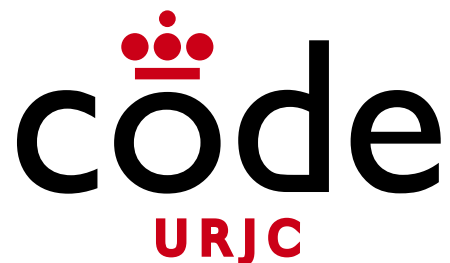


Fundamentos de la Web

Bloque IV: Tecnologías de interactividad en el cliente web

Tema 4.2: AJAX



©2024

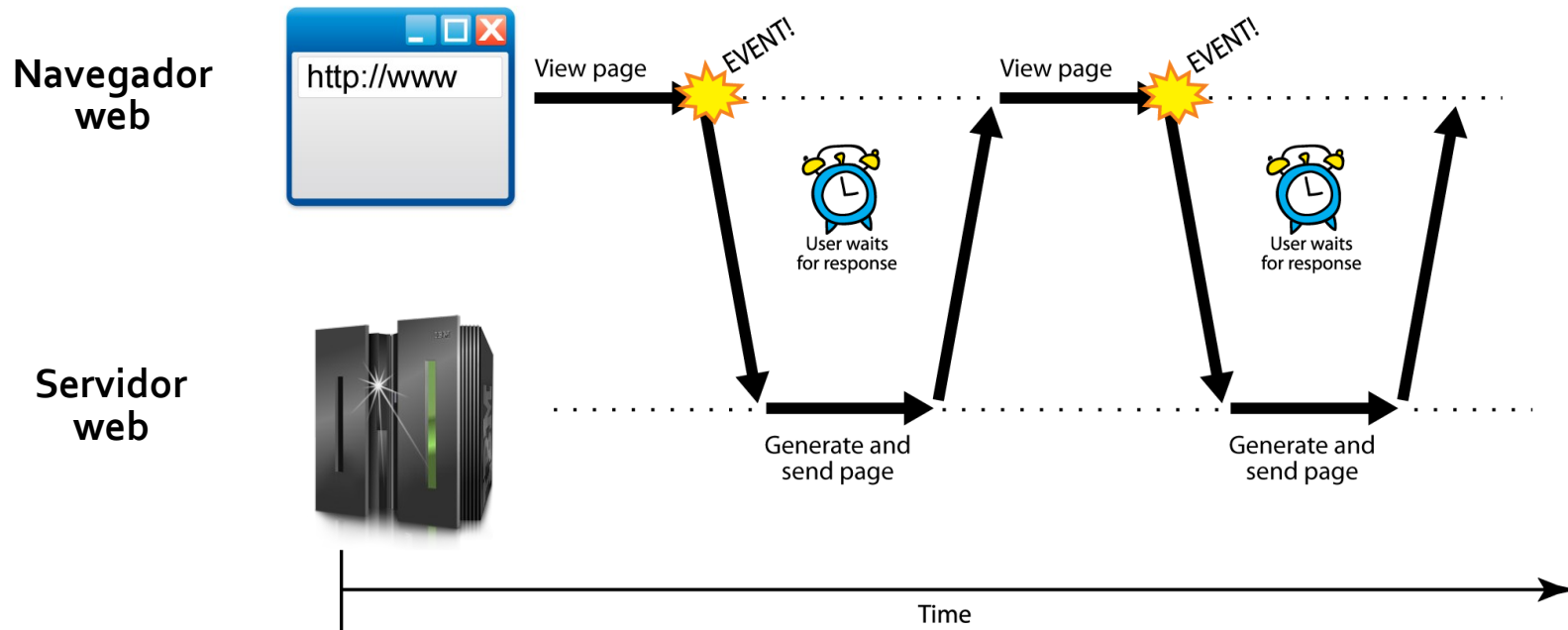
Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

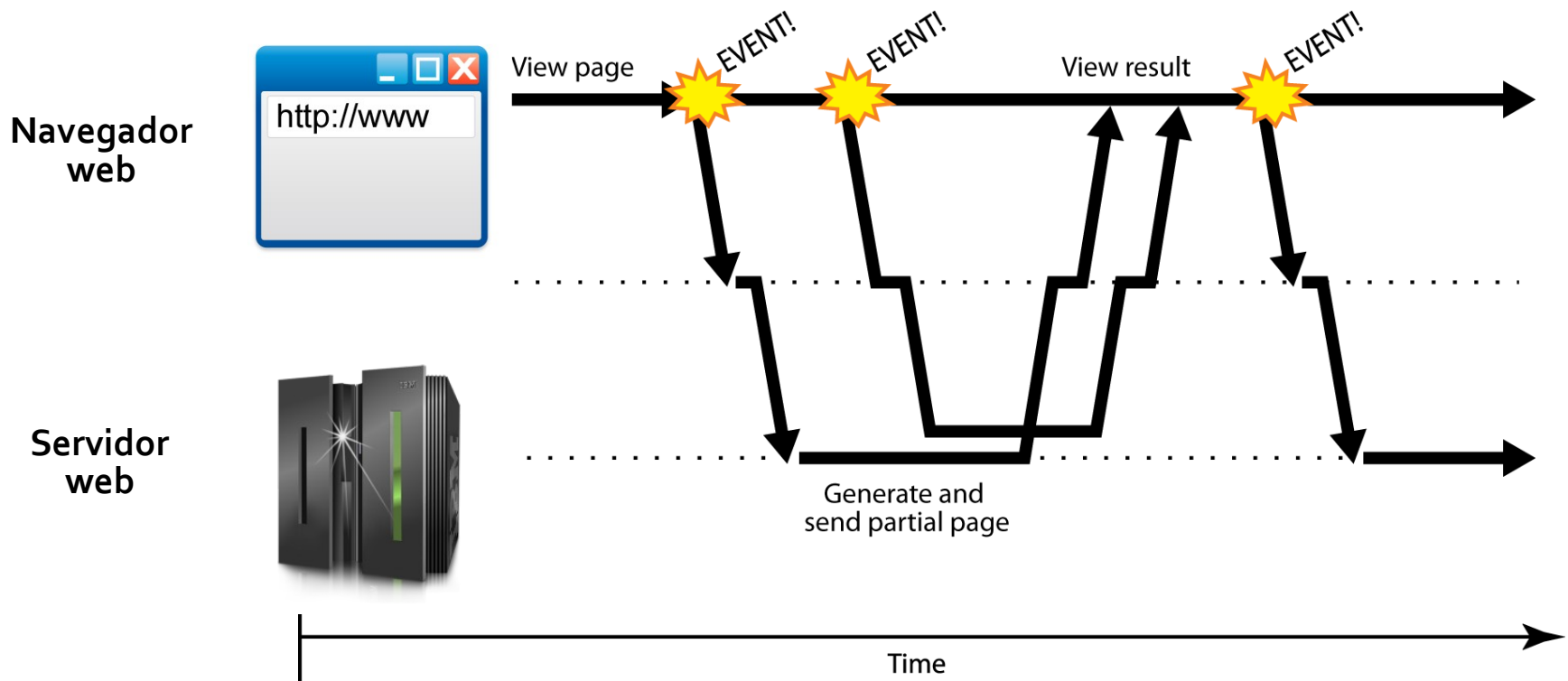
Introducción

- En una página web básica, cada vez que el usuario **pulsa un enlace**, en navegador **recarga completamente la página**

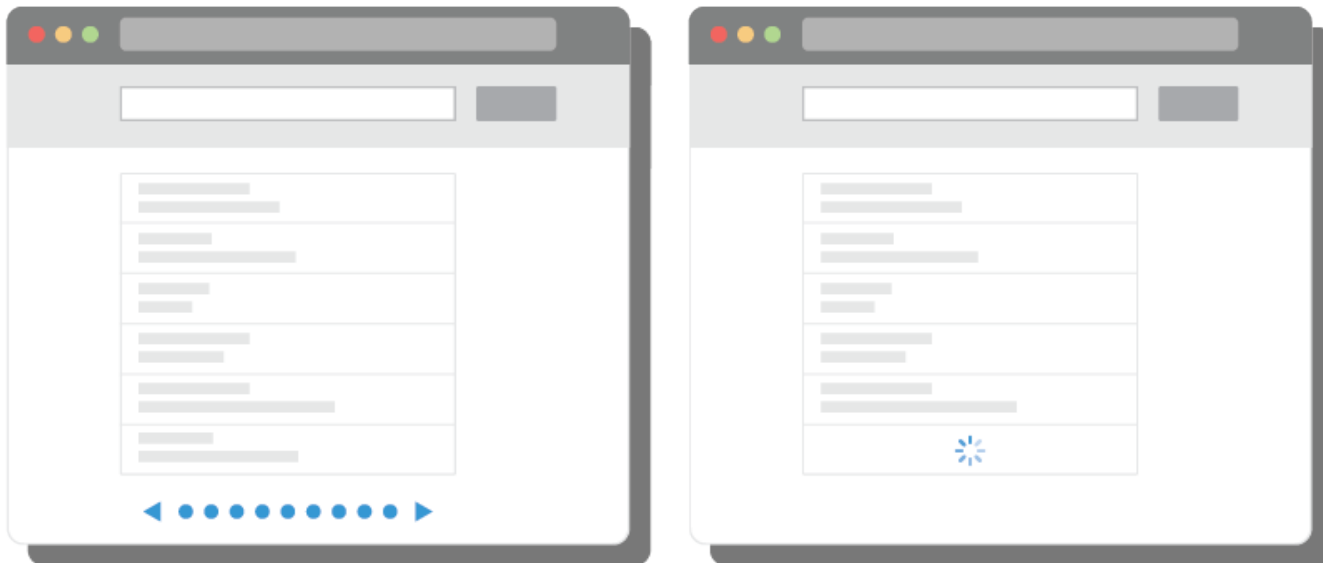


- A veces sólo cambia **parte de la página**, pero se tiene que recargar completa (más **transferencia de datos**, mayor **tiempo de espera...**)
- **AJAX** (*Asynchronous JavaScript And XML*) es una técnica que permite actualizar únicamente parte de la página
- Desde código JavaScript se **solicita al servidor la parte que cambia y actualiza** la página con el contenido obtenido del servidor

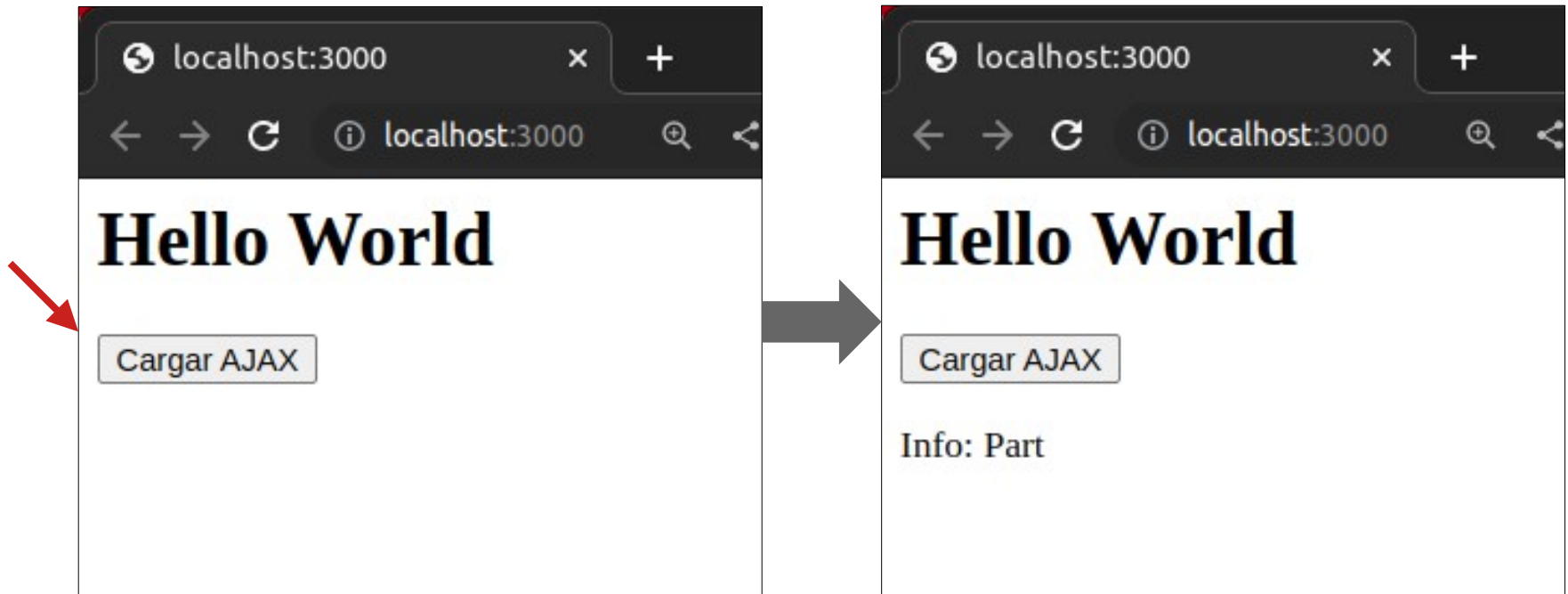
- *AJAX (Asynchronous JavaScript And XML)*



- Es la técnica que se usa para **cargar más resultados** en las búsquedas sin recargar la página (botón “**Cargar más**” o de forma **automática**)



Implementación de AJAX

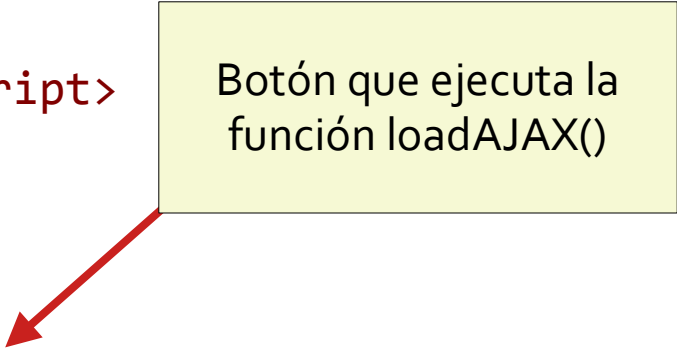


Implementación en cliente

Ejemplo1

- Petición AJAX desde el navegador web

```
<html>
<head>
  <script src="app.js"></script>
</head>
<body>
  <h1>Hello {{name}}</h1>
  <button onclick="loadAJAX()">Cargar AJAX</button>
  <div id="content"></div>
</body>
</html>
```



Botón que ejecuta la función loadAJAX()

Implementación en cliente

Ejemplo1

- Petición AJAX desde el navegador web

```
async function loadAJAX(){  
  
    const response = await fetch('/pagePart');  
    const pagePart = await response.text();  
  
    const content = document.getElementById("content");  
    content.innerHTML = pagePart;  
  
}
```

Implementación en cliente

Ejemplo1

- Petición AJAX desde el navegador web

Hace la petición al servidor de la ruta **/pagePart** en segundo plano y devuelve una respuesta (response)

```
async function loadAJAX(){  
  
    const response = await fetch('/pagePart');  
    const pagePart = await response.text();  
  
    const content = document.getElementById("content");  
    content.innerHTML = pagePart;  
  
}
```

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

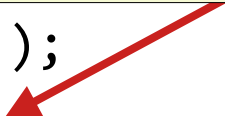
Implementación en cliente

Ejemplo1

- Petición AJAX desde el navegador web

```
async function loadAJAX(){  
  
    const response = await fetch('/pagePart');  
    const pagePart = await response.text();  
  
    const content = document.getElementById("content");  
    content.innerHTML = pagePart;  
  
}
```

Procesa la respuesta y
obtiene el **texto** que
contiene



<https://developer.mozilla.org/en-US/docs/Web/API/Response/text>

Implementación en cliente

Ejemplo1

- Petición AJAX desde el navegador web

```
async function loadAJAX(){  
  
    const response = await fetch('/pagePart');  
    const pagePart = await response.text();  
  
    const content = document.getElementById("content");  
    content.innerHTML = pagePart;  
  
}
```

Se inserta el
contenido cargado
en la página



Implementación en servidor

Ejemplo1

- Generación de contenido AJAX en el servidor

router.js

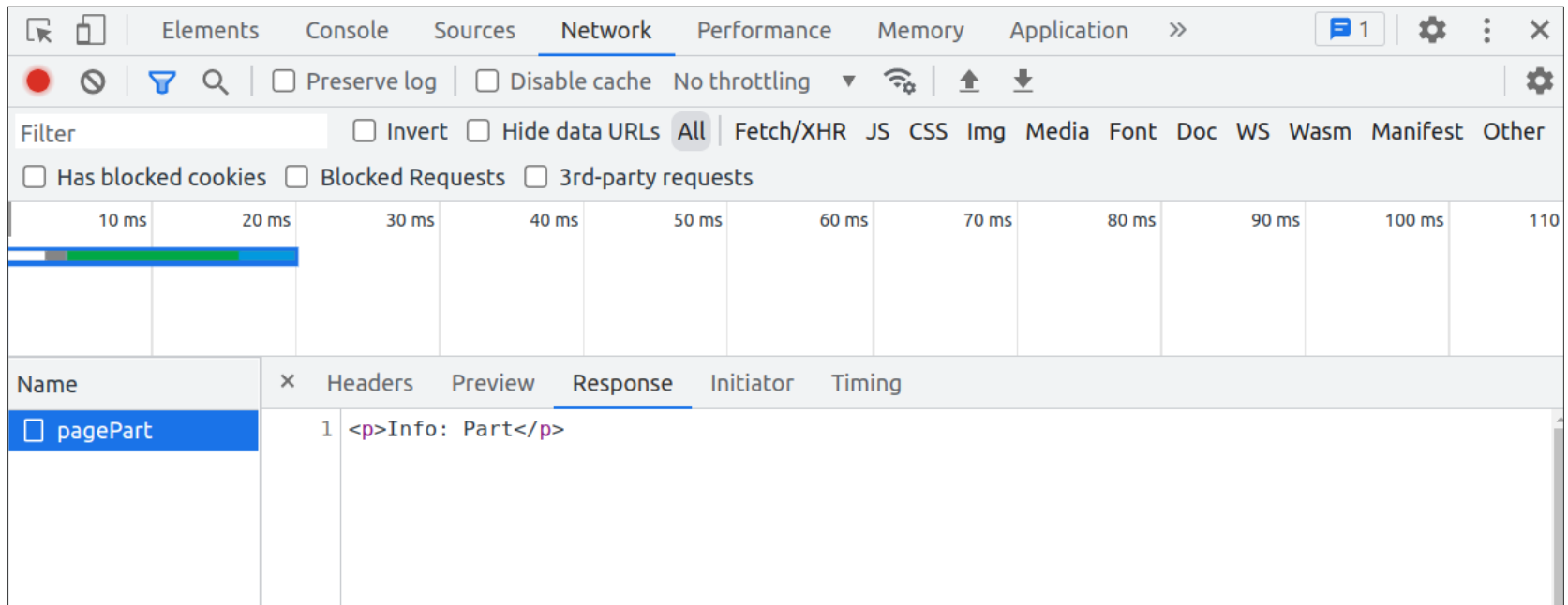
```
import express from 'express';
const router = express.Router();
router.get('/', (req, res) => {
  res.render('index', {
    name: "World"
  });
});
router.get('/pagePart', (req, res) => {
  res.render('pagePart', {
    info: "Part"
  });
});
export default router;
```

pagePart.mustache

<p>Info: {{info}}</p>

Se genera el
contenido igual
que las páginas
completas

- La pestaña “Network/Red” del navegador permite **ver las peticiones** realizadas por AJAX



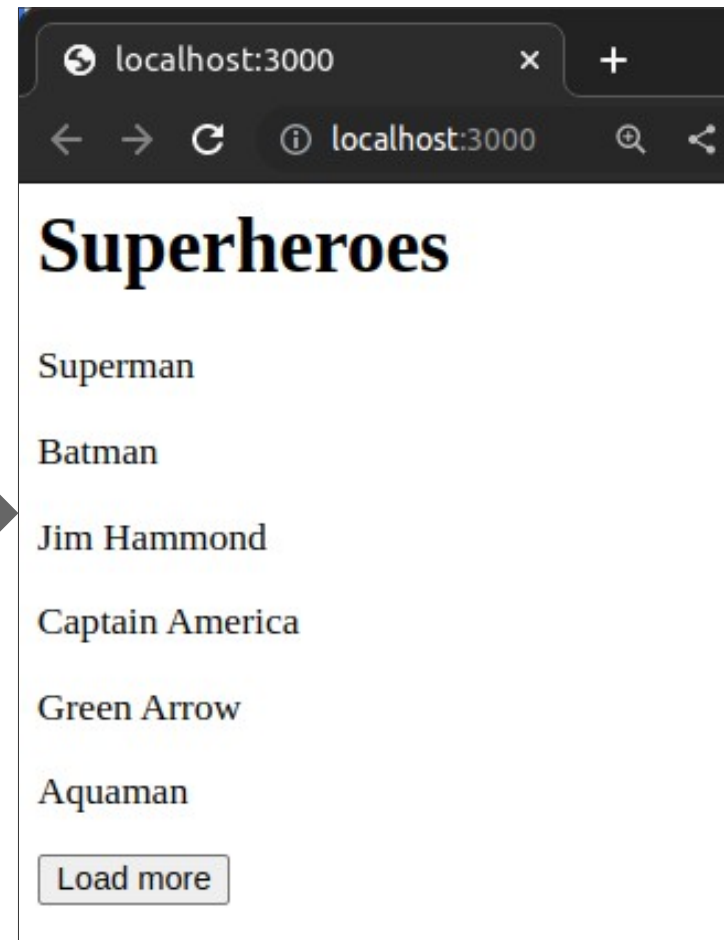
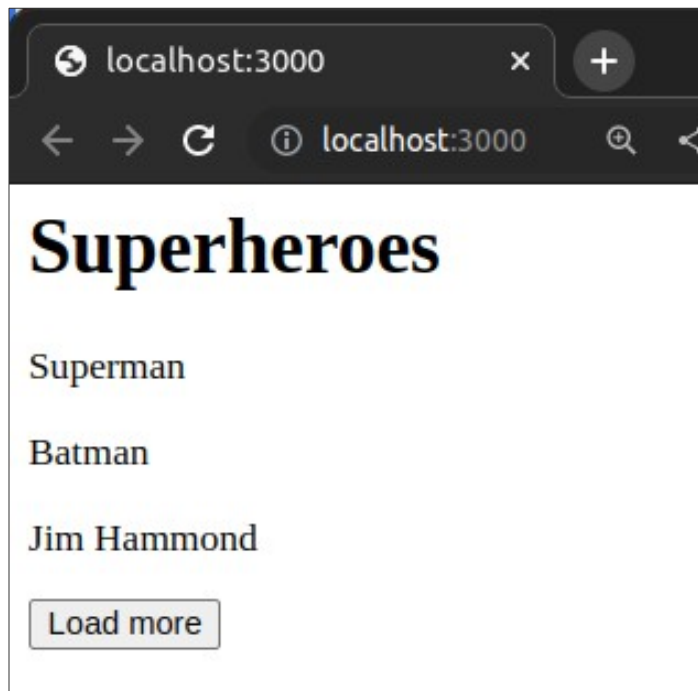
Ejercicio 1

Ejercicio1

- Se tiene una web que muestra los superhéroes guardados en un mapa
- Se quiere implementar un botón de “Cargar más” con AJAX
- Los superhéroes se mostrarán de 3 en 3

Ejercicio 1

Ejercicio1



Ejercicio 1

Ejercicio1

- Al cargar la página se mostrarán los superhéroes 0,1 y 2.
- Cuando se pulse el botón la primera vez se cargarán los superhéroes 3, 4 y 5.
- Cuando se pulse el botón la segunda vez se cargarán los superhéroes 6, 7 y 8.
- La URL que se usa para cargar más resultados deberá indicar qué resultados cargar

Ejercicio 1

Ejercicio1

- Se proporciona una web en la que se muestran inicialmente todos los superhéroes

```
import express from 'express';
import { getSuperheros } from './superheros.js';

const router = express.Router();

router.get('/', (req, res) => {
  const superheroes = getSuperheros();

  res.render('index', {
    superheroes: superheroes
  });
});

export default router;
```

Ejercicio 1

Ejercicio1

- Se proporciona una web en la que se muestran inicialmente todos los superheroes

```
const superheroes = new Map();
let id = 0;

export function addSuperhero(superhero) {
  superheroes.set(id, superhero);
  superhero.id = id;
  id++;
}

export function getSuperhero(id) {
  return superheroes.get(id);
}
```

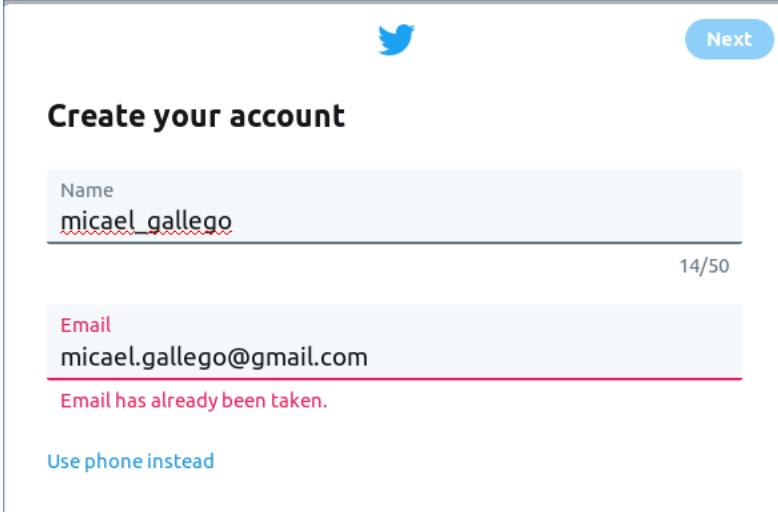
```
export function getSuperheros(from, to) {
  let values = [...superheroes.values()];
  if (from !== undefined) {
    return values.slice(from, to);
  } else {
    return values;
  }
}

export function loadSampleData() {
  addSuperhero({ name: 'Superman' });
  addSuperhero({ name: 'Batman' });
  ...
}

loadSampleData();
```

Información estructurada con AJAX

- AJAX también se puede utilizar para que el código JavaScript **consulte información al servidor**
- Con esa información puede **manipular la página** como quiera (por ejemplo mostrando una alerta)



The screenshot shows a Twitter account creation interface. At the top right is a blue 'Next' button. Below it is the title 'Create your account'. There are two input fields: 'Name' with the value 'michael gallego' and a character count of '14/50'; and 'Email' with the value 'michael.gallego@gmail.com'. A red error message 'Email has already been taken.' is displayed below the email field. At the bottom, there is a link that says 'Use phone instead'.

- Cuando el código **JavaScript** hace peticiones, el servidor puede devolver:

Fragmentos de HTML

Se incrusta directamente en la página

Ej: Cargar más

Información estructurada

Se interpreta por JavaScript para modificar la página

Ej: Error de validación

Información estructurada con AJAX

- Cuando se solicita información estructurada al servidor la suele generar en **formato JSON**
- También se puede devolver en **formato XML**

```
{  
  validation:[  
    {id:"name",status:"ok"},  
    {id:"email",status:"error",message:"Invalid format"}  
  ]  
}
```

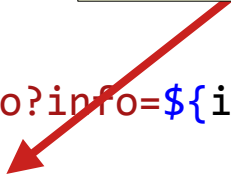
- Cliente

```
async function checkInfo(){  
  
    const info = 'someInfo';  
  
    const response = await fetch(`/checkInfo?info=${info}`);  
  
    const infoCheck = await response.json();  
    const content = document.getElementById("content");  
  
    content.innerHTML = `  
        <p>Valid: ${infoCheck.valid}</p>  
        <p>Message: ${infoCheck.message}</p>`;  
  
}
```

- Cliente

```
async function checkInfo(){  
  const info = 'someInfo';  
  
  const response = await fetch(`/checkInfo?info=${info}`);  
  
  const infoCheck = await response.json();  
  const content = document.getElementById("content");  
  
  content.innerHTML = `  
    <p>Valid: ${infoCheck.valid}</p>  
    <p>Message: ${infoCheck.message}</p>`;  
}
```

La información se
carga en un objeto con
response.json()



- Servidor

```
import express from 'express';
const router = express.Router();
...
router.get('/checkInfo', (req, res) => {
  let info = req.query.info;
  let response = {
    valid: false,
    message: `Info '${info}' not valid`
  }
  res.json(response);
});
export default router;
```

- Servidor

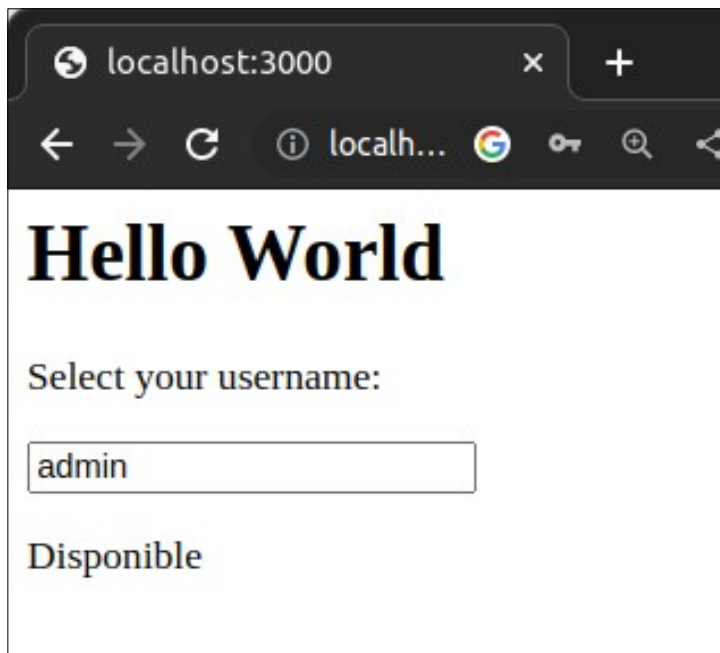
```
import express from 'express';
const router = express.Router();
...
router.get('/checkInfo', (req, res) => {
  let info = req.query.info;
  let response = {
    valid: false,
    message: `Info '${info}' not valid`
  }
  res.json(response);
});
export default router;
```

La información se envía
con `res.json()`



Ejercicio 2

- Implementar una web con un input de texto que consulte la disponibilidad al servidor según vaya escribiendo el nombre de un usuario



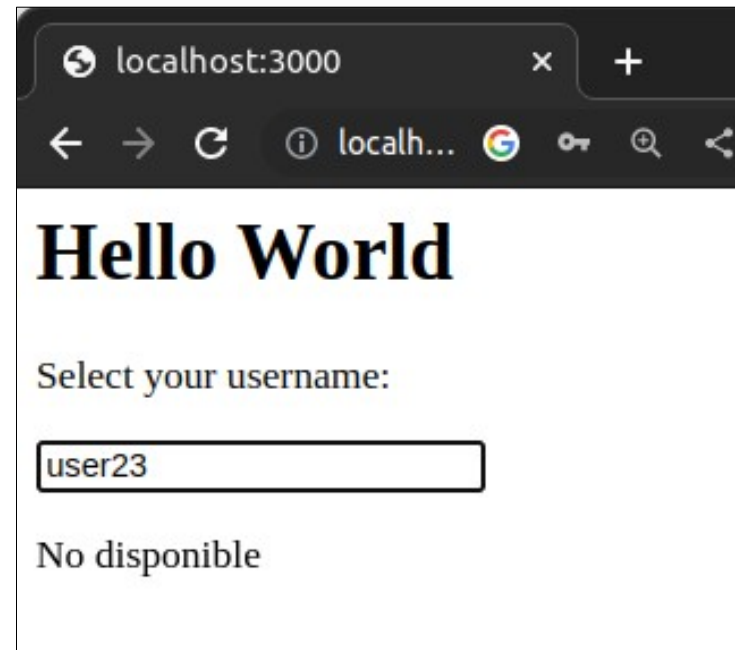
localhost:3000

← → ↻ ⓘ localh... 🔑 🔍 ↶

Hello World

Select your username:

Disponible



localhost:3000

← → ↻ ⓘ localh... 🔑 🔍 ↶

Hello World

Select your username:

No disponible

Ejercicio 2

- El evento “input” permite ejecutar una función cada vez que el texto cambia

```
<input id='username' oninput='checkUsernameAvailability()' type='text'>
```

- Nota: En las web reales sólo se hace la petición cuando el usuario deja de escribir durante un tiempo para no saturar al servidor

Información estructurada con AJAX

Se puede enviar JSON desde el cliente y que sea interpretado por el servidor

Ejemplo3

- Cliente

```
async function applyUppercase() {  
  const text = document.getElementById("text").value;  
  
  const response = await fetch(`/textToUppercase`, {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({  
      text: text,  
    }),  
  });  
  
  const textData = await response.json();  
  const content = document.getElementById("content");  
  
  content.innerHTML += textData.text + " " + textData.textUppercase + "<br>";  
}
```

- Servidor

```
import express from 'express';

const router = express.Router();

...

router.post("/textToUppercase", (req, res) => {
  let textData = req.body;
  let text = textData.text;

  let response = {
    text: text,
    textUppercase: text.toUpperCase(),
  };

  res.json(response);
});

export default router;
```



La información se envía
con `res.json()`

- Servidor

```
import express from 'express';  
  
...  
app.use(express.urlencoded({ extended: true }));  
app.use(express.json());  
  
...
```

Middleware necesario
para interpretar JSON

Envío de formularios por AJAX

- Es posible enviar la información ingresada en un formulario a través de AJAX
- Esto permite un mayor control sobre el procesamiento de la información y ofrece una mejor experiencia al usuario
- Se pueden usar cuadros de diálogo en lugar de páginas de aviso
 - Si el envío del formulario es correcto, se puede insertar el resultado en el HTML sin recargar la página completamente
 - Si ocurre un error, se puede mostrar un mensaje (alert o modal), aceptar y luego editar el formulario sin perder los datos introducidos
- Dependiendo de si el formulario incluye o no ficheros, el envío será diferente

Formulario sin ficheros

- HTML

```
<html>
  <head>
    <script src="app.js"></script>
  </head>

  <body>
    <h1>Procesar el nombre y los apellidos de un usuario</h1>

    <form onsubmit="processUserData(event)">
      <label for="userName">Escribe tu nombre:</label>
      <input type="text" name="userName" required /><br /><br />

      <label for="userSurname">Escribe tu apellido:</label>
      <input type="text" name="userSurname" required /><br /><br />

      <button type="submit">Procesar</button>
    </form>

    <h2>Resultado</h2>
    <div id="content"></div>
  </body>
</html>
```

Interceptar y manejar el envío del formulario en la función **processUserData** al pulsar el botón **Procesar**

Formulario sin ficheros

- HTML

```
<html>
  <head>
    <script src="app.js"></script>
  </head>

  <body>
    <h1>Procesar el nombre y los apellidos de un usuario</h1>

    <form onsubmit="processUserData(event)">
      <label for="userName">Escribe tu nombre:</label>
      <input type="text" name="userName" required /><br /><br />

      <label for="userSurname">Escribe tu apellido:</label>
      <input type="text" name="userSurname" required /><br /><br />

      <button type="submit">Procesar</button>
    </form>

    <h2>Resultado</h2>
    <div id="content"></div>
  </body>
</html>
```

El objeto **event** que se pasa como argumento contiene la información sobre el evento asociado al formulario

Formulario sin ficheros

- Cliente

```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}


```

Se evita el envío
predeterminado (y, por lo tanto,
la recarga de la página)

Formulario sin ficheros

- Cliente

Se obtiene la información del formulario



```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}


```

Formulario sin ficheros

- Cliente

Envío de los datos del formulario en segundo plano a la ruta **"/processUserData"**

```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}


```

Formulario sin ficheros

- Cliente

Se especifica que el formulario **no incluye** ficheros

```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}


```

Formulario sin ficheros

- Cliente

Se procesa la respuesta en formato JSON y se muestra en la página

```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}


```

Formulario sin ficheros

- Cliente

Se eliminan todos los datos introducidos en el formulario

```
async function processUserData(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/processUserData`, {  
    method: "POST",  
    body: new URLSearchParams(formData),  
  });  
  
  const userData = await response.json();  
  
  const content = document.getElementById("content");  
  content.innerHTML = `

Name: ${userData.userName}</p>`;  
  content.innerHTML += `

Username: ${userData.userSurname}</p>`;  
  
  event.target.reset();  
}

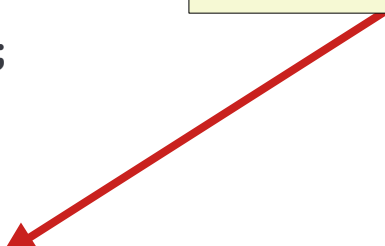

```


Formulario sin ficheros

- Servidor

```
import express from "express";  
const router = express.Router();  
router.get("/", (req, res) => {  
  res.render("index");  
});  
router.post("/processUserData", (req, res) => {  
  let { userName, userSurname } = req.body;  
  
  let response = {  
    userName: userName.toUpperCase(),  
    userSurname: userSurname.toUpperCase(),  
  };  
  
  res.json(response);  
});  
export default router;
```

Se obtiene el nombre y el apellido, se transforman a mayúsculas y se devuelven como un objeto **JSON**



Formulario con ficheros

- Partiendo del Ejemplo 8 – Subida de imágenes del Tema 3.4.1
- Se implementará la subida de imágenes utilizando AJAX
- Los mensajes de confirmación y error se mostrarán mediante un alert

Formulario con ficheros

Sin AJAX

Envío del formulario
sin imagen

Image file

No file chosen

Envío del formulario
con imagen

File not found!

The image 23d841c79022c00602528ec3feec0aab has been uploaded
[View image](#)

Redirección después de pulsar el
enlace "View image"

Image 23d841c79022c00602528ec3feec0aab



Formulario con ficheros

Con AJAX

Envío del formulario
sin imagen

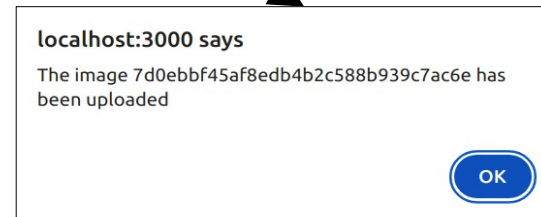


Image file

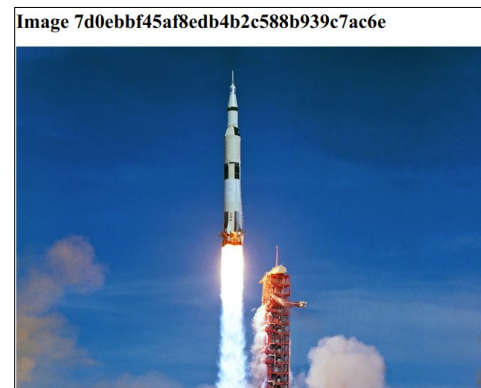
Choose File No file chosen

Save

Envío del formulario
con imagen



Redirección después de pulsar OK



Formulario con ficheros

- HTML

```
<html>
  <head>
    <script src="app.js"></script>
  </head>

  <body>
    <form onsubmit="uploadImage(event)">
      <p>Image file</p>
      <input type='file' name='image' accept=".jpg, .jpeg" />

      <br>
      <input type='submit' value='Save' />
    </form>
  </body>
</html>
```

Interceptar y manejar el envío
del formulario en la función
uploadImage al pulsar el botón
Save



Formulario con ficheros

- Cliente

En general, el código es muy similar al ejemplo de envío de formulario sin ficheros

```
async function uploadImage(event) {
  event.preventDefault();

  const formData = new FormData(event.target);
  const response = await fetch(`/upload_image`, {
    method: "POST",
    body: formData,
  });

  const result = await response.json();

  if (result.valid) {
    alert(result.message);
    window.location = "/view_image";
  } else {
    alert(`Error: ${result.message}`);
  }
}
```

Formulario con ficheros

- Cliente

Se especifica que el formulario **sí incluye** ficheros

```
async function uploadImage(event) {  
  event.preventDefault();  
  
  const formData = new FormData(event.target);  
  const response = await fetch(`/upload_image`, {  
    method: "POST",  
    body: formData,  
  });  
  
  const result = await response.json();  
  
  if (result.valid) {  
    alert(result.message);  
    window.location = "/view_image";  
  } else {  
    alert(`Error: ${result.message}`);  
  }  
}
```

Formulario con ficheros

- Cliente

Si la imagen se sube correctamente, se muestra un mensaje en un cuadro de diálogo. Al cerrarlo, el usuario es redirigido a la página de visualización

```
async function uploadImage(event) {
  event.preventDefault();

  const formData = new FormData(event.target);
  const response = await fetch(`/upload_image`, {
    method: "POST",
    body: formData,
  });

  const result = await response.json();

  if (result.valid) {
    alert(result.message);
    window.location = "/view_image";
  } else {
    alert(`Error: ${result.message}`);
  }
}
```


Formulario con ficheros

- Cliente

Si existe un error al subir la imagen, se muestra un cuadro de diálogo. Al cerrarlo, el usuario podrá continuar modificando el formulario

```
async function uploadImage(event) {
  event.preventDefault();

  const formData = new FormData(event.target);
  const response = await fetch(`/upload_image`, {
    method: "POST",
    body: formData,
  });

  const result = await response.json();

  if (result.valid) {
    alert(result.message);
    window.location = "/view_image";
  } else {
    alert(`Error: ${result.message}`);
  }
}
```

Formulario con ficheros

- Servidor

```
import express from "express";
import multer from "multer";

const router = express.Router();

const upload = multer({ dest: "uploads/" });

let filename;

router.post("/upload_image", upload.single("image"), (req, res) => {
  let response = { valid: false, message: "" };

  if (req.file) {
    filename = req.file.filename;
    response.valid = true;
    response.message = `The image ${filename} has been uploaded`;
  } else {
    response.message = "File not found!";
  }

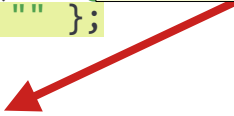
  res.json(response);
});

router.get("/view_image", (req, res) => {
  res.render("view_image", { filename });
});

router.get("/download_image", (req, res) => {
  res.download("uploads/" + filename);
});

export default router;
```

Si el formulario incluye una imagen, se devuelve un objeto JSON con un mensaje de confirmación. Si no, se devuelve un mensaje de error



Ejercicio 3

- Partiendo de la solución del Ejercicio 4 – Tablón con imágenes del Tema 3.4.1
- Realiza las siguientes peticiones utilizando AJAX:
 - Creación de un nuevo anuncio
 - Borrado de un anuncio existente

Conclusiones

- Código JavaScript en el **navegador** puede ejecutar código JavaScript en el **servidor** (haciendo una **petición http**)
- El servidor puede **devolver HTML** (que se incrusta en la página) o **JSON** (que se procesa)